

架构部分

总述

软件架构是什么

结构 (structure) 、元素 (elements) 、关系 (relationships) 、设计 (design)

架构的来源

1. 非功能性需求, NFRs, non-functional requirements
2. 架构攸关需求, ASRs, architecture-significant requirements
3. 质量属性, Quality Requirements
4. 涉众, Stakeholders
5. 组织, Organisations
6. 技术环境, Technical Environment

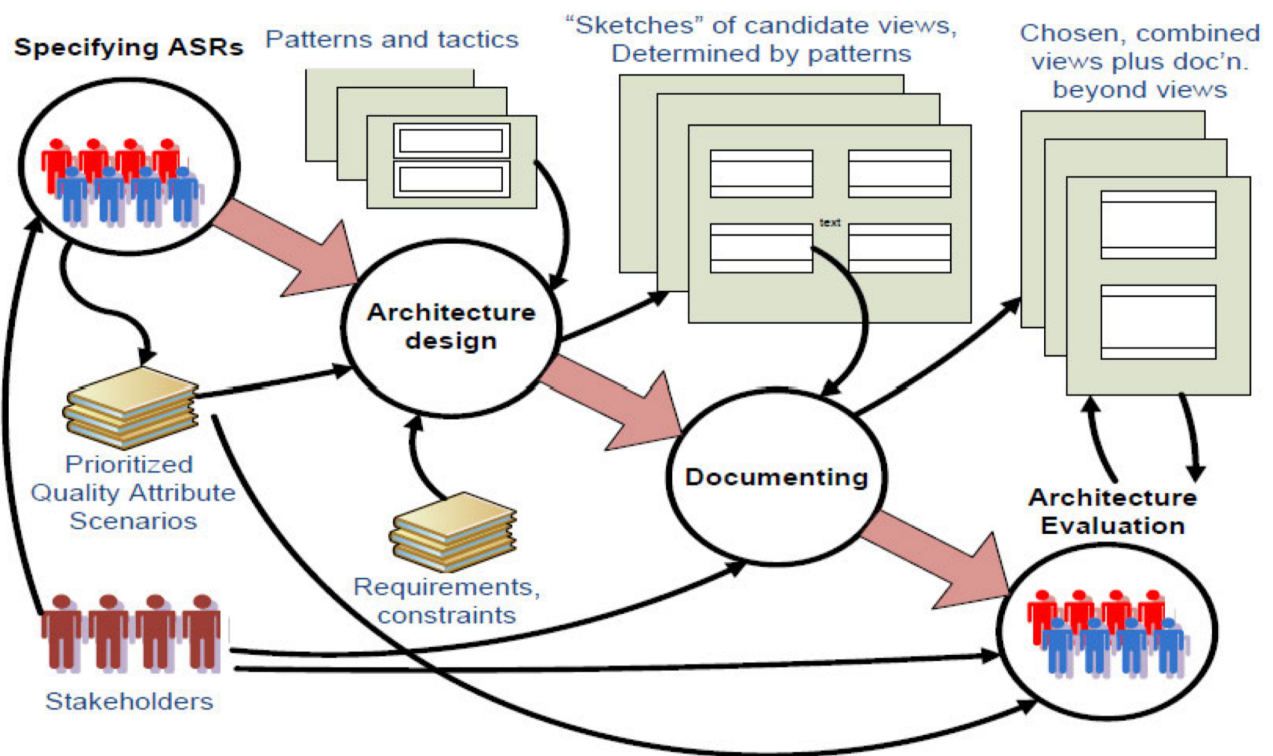
(以下来自 zhy 的往年题整理)

7. 业务目标
8. 商业与技术决策组合

架构（相关）过程 Architecture Process

描述与架构相关的软件生命周期活动。

1. 识别 ASRs
 - 输入：无
 - 输出：排序后的质量属性场景
2. 架构设计
 - 输入：排序后的质量属性场景、需求和约束、模式和决策 (tactics)
 - 输出：一组候选视图的草图（由模式决定）
3. 架构文档化
 - 输入：一组候选视图的草图（由模式决定）
 - 输出：View & Beyond
4. 架构评估
 - 输入：View & Beyond、优化的质量属性场景
 - 输出：View & Beyond



补充

- 如果在 4. 架构评估 中发现遗漏了 ASR，则回到 1. 识别 ASR
- 如果在 4. 架构评估 中发现不满足 ASR，则回到 2. 架构设计

质量属性 Quality Attributes

质量属性可以被分为内部质量属性和外部质量属性：

- 内部：和开发人员有关，使系统更容易开发、维护，比如：可修改性、可维护性
- 外部：和用户、使用者（不了解系统具体实现）有关，比如：性能，可用性 Availability

如何描述质量属性场景？

- 刺激源 Source：产生刺激的实体（人，系统或任何其他触发），可能是输入、信息等，对当前的状态的一个变化。
- 刺激 Stimulus：到达系统时需要考虑的条件。
- 响应 Response：刺激到来后工件开展的行为。
- 响应度量 Response Measure：对刺激的响应以某种方法进行测量，以便可以测试需求（比如多长时间系统有反馈）。
- 环境 Enviroment：发生刺激时系统的状况，例如系统正常运行、系统过载、系统受到攻击、系统网络出现故障等。
- 工件 Artifact：完成需求的整个系统或者系统的一部分（软件制品）。

质量属性	刺激源	刺激	工件	环境	响应	响应度量
Availability	Heartbeat 监视器	服务器无响应	处理器	正常操作	通知操作者继续操作	没有停机时间
Interoperability	车辆信息系统	发送当前位置	路况监控系统	系统运行前已知	路况监控结合当前位置和 Google 地图上的其他信息，并且进行广播	我们的信息在 99.9%的时间是正确的
Modifiability	开发者	希望修改 UI 界面	代码	设计时	进行修改和单元测试	在 3 个小时内完成
Performance	用户	发起事务	系统	正常操作	事务被处理	平均延迟不超过 2 秒
Security	来自偏远地区心怀不满的员工	尝试修改支付率	系统内数据	正常操作	系统存储修改追踪	正确数据在 1 天内被存储并且进行篡改身份识别
Testability	单元测试者	单元测试完成	单元测试代码	开发时	记录结果	3 小时内达到 85%的路径覆盖
Usability	用户	下载新应用	系统	运行时	用户高效地使用应用	在 2 分钟以内的实验

架构模式 Architecture Patterns

1. 架构模式的要素

- 上下文 Context
- 问题 Problem
- 解决方案 Solution

如果没有 solution，就变成了架构的特点。

2. 架构模式的分类

- 模块化模式 Module Patterns
 - 特指开发态（静态）
 - 例如：分层模式 layered pattern、微内核模式 micro-kernel pattern
- 组件-连接器模式 Component-Connector Patterns
 - 倾向于描述运行态（动态）
 - 其中 Component 可以连接 1 对多的关系，Connector 只能连接一对一的关系
 - 例如：中继器模式 Broker Pattern、MVC 模式、管道-过滤器模式 Pipe-Filter Pattern、客户端-服务端 client-server 模式、P2P 模式、服务优先模式 service-oriented pattern、发布-订阅模式 publish-subscribe pattern、共享数据模式 share-data pattern、SOA 模式
- 分配模式 Allocation Patterns
 - 把软件的元素部署/分配到非软件的环境当中

◦ 比如：

- Map-reduce Pattern: 充分引入并行，在不同计算资源上部署
- Multi-tier Pattern: 把一组运行在相同计算资源上的元素进行不同的组合（逻辑关系）

架构设计 Designing Architecture

设计策略

- 抽象 Abstraction: 使用抽象让设计师关注本身结构而不关注实现，比如将系统抽象为组件和连接件或抽象为模块
- 分解 Decomposition: 针对某一个系统关注点分解后处理，比如将整个系统分解或将某个模块分解
- 分治 Devide & Conquer: 将某个模块分别处理
- 生成与测试 Generation and Test: 将一个特定的设计看作是一个假设；根据测试路径生成测试用例
- 迭代与细化 Iteration: 使用迭代的方法，ADD 方法多次迭代直到满足所有 ASR
- 复用 Reuse: 重用在设计过程中出现的可以复用的元素，重用现有架构

属性驱动设计 Attribute-Driven Design ADD (2.0)

1. 确认需求 `confirm requirements`
2. 选择一个待分解的元素 `choose an element to decompose`
3. 识别架构攸关需求 `identify ASRs`
4. 选择一种满足 ASR 的设计 `choose a design satisfying ASRs`
 - 4.1. 找出设计问题 `identify concerns`
 - 4.2. 列出子关注点代替模式 / 决策 `list alternatives`
 - 4.3. 从清单中选择模式 / 决策 `select patterns / tactics`
 - 4.4. 确定模式 / 决策与 ASR 之间的关系 `determine relations`
 - 4.5. 记录初步的架构视图 `capture views`
 - 4.6. 评估并解决不一致的问题 `resolve inconsistencies`
5. 实例化架构元素 & 分配职责 `instantiate elements & allocate responsibilities`
6. 定义接口 `define interface`
7. 验证和完善需求 `verify & refine requirements`
8. 重复 2-7 直到所有的 ASR 都被满足 `repeat step 2-7 until all ASRs satisfied`

文档化架构 Documenting Architecture

典型的软件架构文档包中应该包含 **View** 和 **Beyond** 部分。

Beyond 部分

- 文档路线图：包含了范围和总结、简单摘要等
- 视图的文档组织方式：描述了本文档中视图是如何组织的
- 系统概述：从整体上描述了当前架构的简要说明、业务目标（驱动因素）等等
- 视图之间的映射关系：描述了不同视图之间的映射关系
- 系统原理：从整体上描述了当前架构的设计原理
- 目录-索引、词汇表、首字母缩略词表

View 部分

- styles and views (体系结构风格和视图)
- structural views
 - module views
 - C & C views
 - allocation views
- quality views

每一个 View 的内容

- 主要介绍：显示视图的元素和关系，以及图例
- 元素介绍：详细介绍第一部分中描述的元素、元素属性、关系属性和元素接口和行为
- 上下文图：描述系统如何与环境相关
- 可变性指南：告知视图中可能出现的变化
- 基本原理：解释设计如何映射在视图中，以及其合理性

架构评估 Evaluating Architecture

ATAM: Architecture Tradeoff Analysis Method

ATAM 的输入和输出

- Risks(non-risks, risk themes)
- Sentitivity points
- Trade-off points

ATAM 各个过程和其中的涉众

1. 阶段-0：准备和建立团队

- 参与者：评估团队领导和关键项目决策者
- 职责：根据架构设计文档生成评估计划，包括谁参加评估、如何何时何地地开展评估、最后评估报告会被呈递给谁

2. 阶段-1：评估-1

- 参与者：评估团队和项目决策者
- 职责：
 1. 评估负责人介绍 ATAM 方法 `present ATAM`
 2. 项目经理或客户从业务角度介绍业务驱动因素 `present business drivers`
 3. 首席架构师介绍体系结构 `present architecture`
 4. 评估团队确定架构方法 `identify architectural approaches`
 5. 评估团队和项目决策者生成质量属性效用树 (Utility Tree) `generate unility tree`
 6. 评估团队分析架构方法 `analyse architectural approaches`

3. 阶段-2：评估-2

- 参与者：评估团队、项目决策者和项目涉众
- 职责：
 1. 评估负责人介绍 ATAM 方法和之前已经取得的成果 `present ATAM & results`
 2. (6.) 涉众头脑风暴并确定场景优先级 `brainstorm & prioritize`
 3. (7.) 评估团队分析架构方法 (类似 6.) `analyse architectural approaches`
 4. (8.) 评估团队展示评估结果，并呈递给涉众 `present results`

4. 阶段-3：后续 `follow-up`

- 参与者：评估团队、主要涉众
- 职责：评估团队制作最终评估报告，发给主要涉众审核通过后，将报告呈递给委托评估的人。